

Contents

1	Python Interview Questions for Experienced	1
1.1	Questions	1
2	Answers	2
2.1	1. How is memory managed in Python?	2
2.2	2. What are Python namespaces? Why are they used?	2
2.3	3. What is scope resolution in Python?	3
2.4	4. What are decorators in Python?	4
2.5	5. What are dictionary and list comprehensions?	4
	2.5.1 List comprehension	5
	2.5.2 Dictionary comprehension	5
2.6	6. What is lambda in Python? Why is it used?	5
2.7	7. How do you copy an object in Python?	6
	2.7.1 Shallow copy	6
	2.7.2 Deep copy	6
2.8	8. What is the difference between <code>xrange()</code> and <code>range()</code> ?	6
2.9	9. What are pickling and unpickling?	7
2.10	10. What are generators in Python?	7
2.11	11. What is <code>PYTHONPATH</code> in Python?	8
2.12	12. What are <code>help()</code> and <code>dir()</code> used for?	8
	2.12.1 <code>help()</code>	8
	2.12.2 <code>dir()</code>	9
2.13	13. What is the difference between <code>.py</code> and <code>.pyc</code> files?	9
2.14	14. How is Python interpreted?	9
2.15	15. How are arguments passed in Python?	10
2.16	16. What are iterators in Python?	11
2.17	17. How do you delete a file in Python?	11
2.18	18. What are <code>split()</code> and <code>join()</code> in Python?	12
	2.18.1 <code>split()</code>	12
	2.18.2 <code>join()</code>	12
2.19	19. What are <code>*args</code> and <code>**kwargs</code> ?	12
2.20	20. What are negative indexes and why are they used?	13
3	Rapid Revision	13

1 Python Interview Questions for Experienced

1.1 Questions

1. How is memory managed in Python?
2. What are Python namespaces? Why are they used?
3. What is scope resolution in Python?
4. What are decorators in Python?
5. What are dictionary and list comprehensions?
6. What is lambda in Python? Why is it used?
7. How do you copy an object in Python?

8. What is the difference between `xrange()` and `range()`?
 9. What are pickling and unpickling?
 10. What are generators in Python?
 11. What is `PYTHONPATH` in Python?
 12. What are `help()` and `dir()` used for?
 13. What is the difference between `.py` and `.pyc` files?
 14. How is Python interpreted?
 15. How are arguments passed in Python?
 16. What are iterators in Python?
 17. How do you delete a file in Python?
 18. What are `split()` and `join()` in Python?
 19. What are `*args` and `**kwargs`?
 20. What are negative indexes and why are they used?
-

2 Answers

2.1 1. How is memory managed in Python?

Python memory is managed automatically by the **Python Memory Manager**.

Python objects are stored in a private heap, and memory cleanup is handled automatically using mechanisms such as:

- Reference counting
- Garbage collection
- Python's internal memory allocator

Developers normally do not manually allocate and free memory.

Interview answer: Python automatically manages memory through its memory manager, reference counting, and garbage collection.

[Back to Questions](#)

2.2 2. What are Python namespaces? Why are they used?

A namespace maps names to objects.

Conceptually:

`name` → `object`

Python uses namespaces to avoid naming conflicts.

Common namespaces include:

- Local namespace
- Global namespace
- Built-in namespace

Example:

```

x = 10

def test():
    x = 20
    print(x)

test()
print(x)

```

Output:

```

20
10

```

The two `x` variables belong to different namespaces.

Interview answer: A namespace is a mapping between names and objects. Namespaces prevent naming conflicts by allowing the same name to exist in different scopes.

[Back to Questions](#)

2.3 3. What is scope resolution in Python?

Scope resolution determines which variable Python uses when the same name exists in multiple scopes.

Python follows the **LEGB** rule:

```

Local
↓
Enclosing
↓
Global
↓
Built-in

```

Example:

```

x = 10

def test():
    x = 20
    print(x)

test()
print(x)

```

Output:

```

20
10

```

The local `x` takes precedence inside the function.

Interview answer: Python resolves variable names using the LEGB rule: Local, Enclosing, Global, and Built-in.

[Back to Questions](#)

2.4 4. What are decorators in Python?

A decorator adds or modifies the behavior of a function without changing the function's original code.

Decorators use the `@` syntax.

```
def logger(func):
    def wrapper():
        print("Function started")
        func()
    return wrapper
```

```
@logger
def greet():
    print("Hello")
```

```
greet()
```

Output:

```
Function started
Hello
```

Common uses include:

- Logging
- Authentication
- Caching
- Timing
- Validation

Interview answer: A decorator wraps another function or method to extend its behavior without modifying its original implementation.

[Back to Questions](#)

2.5 5. What are dictionary and list comprehensions?

Comprehensions provide concise syntax for creating collections.

2.5.1 List comprehension

```
numbers = [1, 2, 3, 4]
```

```
squares = [x * x for x in numbers]
```

Result:

```
[1, 4, 9, 16]
```

2.5.2 Dictionary comprehension

```
squares = {x: x * x for x in range(4)}
```

Result:

```
{0: 0, 1: 1, 2: 4, 3: 9}
```

Filtering is also possible:

```
even = [x for x in range(10) if x % 2 == 0]
```

Interview answer: Comprehensions are concise constructs for generating lists, dictionaries, or sets from iterable data, optionally with filtering.

[Back to Questions](#)

2.6 6. What is lambda in Python? Why is it used?

A lambda is a small anonymous function.

Syntax:

```
lambda arguments: expression
```

Example:

```
multiply = lambda a, b: a * b
```

```
print(multiply(2, 5))
```

Output:

```
10
```

A common use is sorting:

```
users = [  
    {"name": "Alice", "age": 30},  
    {"name": "Bob", "age": 25}  
]
```

```
users.sort(key=lambda user: user["age"])
```

Interview answer: A lambda is an anonymous function containing a single expression. It is commonly used for short operations passed to functions such as `sorted()`, `map()`, or `filter()`.

[Back to Questions](#)

2.7 7. How do you copy an object in Python?

Assignment does not copy an object.

```
a = [1, 2, 3]
b = a
```

Both names reference the same list.

Python's `copy` module provides:

- Shallow copy
- Deep copy

2.7.1 Shallow copy

```
import copy
```

```
original = [[1, 2], [3, 4]]
copied = copy.copy(original)
```

Nested objects are still shared.

2.7.2 Deep copy

```
import copy
```

```
copied = copy.deepcopy(original)
```

Nested objects are copied recursively.

Interview answer: Use `copy.copy()` for a shallow copy and `copy.deepcopy()` for a recursive deep copy.

[Back to Questions](#)

2.8 8. What is the difference between `xrange()` and `range()`?

This question mainly applies to **Python 2**.

In Python 2:

- `range()` creates a list
- `xrange()` produces values lazily

In Python 3:

`xrange()` does not exist

Python 3's `range()` behaves more like Python 2's `xrange()` and does not create a full list in memory.

Example:

```
for i in range(5):
    print(i)
```

Interview answer: `xrange()` existed in Python 2 as a memory-efficient lazy sequence. In Python 3 it was removed, and `range()` now provides similar lazy behavior.

[Back to Questions](#)

2.9 9. What are pickling and unpickling?

Pickling converts a Python object into a byte stream.

Unpickling converts that byte stream back into a Python object.

Example:

```
import pickle

data = {"name": "Alice", "age": 30}

with open("data.pkl", "wb") as file:
    pickle.dump(data, file)
```

Read it back:

```
with open("data.pkl", "rb") as file:
    data = pickle.load(file)
```

Important: Never unpickle data from an untrusted source.

Interview answer: Pickling serializes Python objects into bytes, while unpickling reconstructs the objects from those bytes.

[Back to Questions](#)

2.10 10. What are generators in Python?

Generators produce values one at a time instead of storing all values in memory.

They use `yield`.

```
def numbers():
    yield 1
    yield 2
    yield 3
```

```
for number in numbers():  
    print(number)
```

Output:

```
1  
2  
3
```

Generators are useful for:

- Large datasets
- Streams
- Memory-efficient iteration
- Lazy evaluation

Interview answer: A generator is a function that uses `yield` to produce values lazily, one at a time, while preserving its state between iterations.

[Back to Questions](#)

2.11 11. What is PYTHONPATH in Python?

PYTHONPATH is an environment variable that adds directories to Python's module search path.

Example:

```
export PYTHONPATH=/home/user/my_modules
```

Python can then search that directory when importing modules.

You can inspect the current search path using:

```
import sys  
  
print(sys.path)
```

Interview answer: PYTHONPATH allows you to add directories where Python should search for modules and packages.

[Back to Questions](#)

2.12 12. What are `help()` and `dir()` used for?

2.12.1 `help()`

Displays documentation.

```
help(str)
```

2.12.2 `dir()`

Displays attributes and methods available on an object.

```
print(dir(str))
```

Useful during:

- Debugging
- Exploration
- Interactive development

Interview answer: `help()` displays documentation, while `dir()` lists attributes and methods available on an object.

[Back to Questions](#)

2.13 13. What is the difference between `.py` and `.pyc` files?

A `.py` file contains Python source code.

```
app.py
```

A `.pyc` file contains compiled Python bytecode.

```
app.cpython-313.pyc
```

Bytecode files are commonly stored under:

```
__pycache__/
```

They can reduce repeated compilation work for imported modules.

Interview answer: `.py` files contain Python source code, while `.pyc` files contain compiled bytecode used by the Python interpreter.

[Back to Questions](#)

2.14 14. How is Python interpreted?

Using CPython, the process is roughly:

```
.py source code
    ↓
compile
    ↓
bytecode
    ↓
Python Virtual Machine
    ↓
execution
```

Python as a language is not inherently limited to one execution model.

Different implementations include:

- CPython
- PyPy
- Jython
- IronPython

Interview answer: In CPython, source code is compiled into bytecode and then executed by the Python Virtual Machine.

Back to Questions

2.15 15. How are arguments passed in Python?

Python uses **object references**, often described as **call by object sharing**.

Example with a mutable object:

```
def add_item(items):  
    items.append("Python")
```

```
skills = ["SQL"]
```

```
add_item(skills)
```

```
print(skills)
```

Output:

```
['SQL', 'Python']
```

The function receives a reference to the same list object.

However, reassignment behaves differently:

```
def change(value):  
    value = 100
```

```
x = 10
```

```
change(x)
```

```
print(x)
```

Output:

```
10
```

Interview answer: Python passes references to objects. Mutating a passed mutable object can affect the caller, while reassigning the local parameter does not replace the caller's reference.

[Back to Questions](#)

2.16 16. What are iterators in Python?

An iterator is an object that produces values one at a time.

It implements:

```
__iter__()  
__next__()
```

Example:

```
numbers = [10, 20, 30]  
  
iterator = iter(numbers)
```

```
print(next(iterator))  
print(next(iterator))
```

Output:

```
10  
20
```

When no values remain, `StopIteration` is raised.

Interview answer: An iterator is an object that implements `__iter__()` and `__next__()` and returns one element at a time until `StopIteration` is raised.

[Back to Questions](#)

2.17 17. How do you delete a file in Python?

Use `os.remove()`:

```
import os  
  
os.remove("data.txt")
```

Another option is `Path.unlink()`:

```
from pathlib import Path  
  
Path("data.txt").unlink()
```

Interview answer: A file can be deleted using `os.remove()` or `Path.unlink()`.

[Back to Questions](#)

2.18 18. What are `split()` and `join()` in Python?

2.18.1 `split()`

Splits a string into a list.

```
text = "Python SQL Docker"
```

```
skills = text.split()
```

Result:

```
['Python', 'SQL', 'Docker']
```

2.18.2 `join()`

Combines strings using a separator.

```
result = ", ".join(skills)
```

Result:

```
Python, SQL, Docker
```

Interview answer: `split()` converts a string into a list of substrings, while `join()` combines strings from an iterable into one string.

[Back to Questions](#)

2.19 19. What are `*args` and `**kwargs`?

`*args` accepts a variable number of positional arguments.

```
def total(*args):  
    return sum(args)
```

```
print(total(1, 2, 3))
```

Output:

```
6
```

Inside the function, `args` is a tuple.

`**kwargs` accepts a variable number of keyword arguments.

```
def show_user(**kwargs):  
    print(kwargs)
```

```
show_user(name="Alice", age=30)
```

Output:

```
{'name': 'Alice', 'age': 30}
```

Inside the function, `kwargs` is a dictionary.

Interview answer: `*args` collects extra positional arguments into a tuple, while `**kwargs` collects extra keyword arguments into a dictionary.

[Back to Questions](#)

2.20 20. What are negative indexes and why are they used?

Negative indexes access elements from the end of a sequence.

Example:

```
numbers = [10, 20, 30, 40]
```

```
print(numbers[-1])
print(numbers[-2])
```

Output:

```
40
30
```

Index mapping:

10	20	30	40
0	1	2	3
-4	-3	-2	-1

Negative indexes are useful when you want to access values relative to the end without calculating the sequence length.

Interview answer: Negative indexes access sequence elements from the end, where `-1` represents the last element.

[Back to Questions](#)

3 Rapid Revision

Topic	Key Point
Memory	Automatic memory management
Namespace	Maps names to objects
Scope	LEGB
Decorator	Extends function behavior
Comprehension	Concise collection creation
Lambda	Anonymous one-expression function
Shallow copy	Nested references may be shared
Deep copy	Recursively copies nested objects
<code>range()</code>	Lazy range object in Python 3

Topic	Key Point
Pickle	Python object serialization
Generator	Lazy values using <code>yield</code>
<code>PYTHONPATH</code>	Additional module search paths
<code>help()</code>	Documentation
<code>dir()</code>	Attributes and methods
<code>.py</code>	Source code
<code>.pyc</code>	Bytecode
Arguments	Object references / object sharing
Iterator	<code>__iter__()</code> + <code>__next__()</code>
File deletion	<code>os.remove()</code> / <code>Path.unlink()</code>
<code>split()</code>	String $\rightarrow$ list
<code>join()</code>	Iterable of strings $\rightarrow$ string
<code>*args</code>	Extra positional arguments
<code>**kwargs</code>	Extra keyword arguments
Negative index	Access from end